FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

1

RANDOM WALK SIMULATION USING AGENTPY

Student Name	**Kelzo Marcel O. Lumabas**
Section:	**AN42**
Professor:	**Ms. Crisola G. Tan**
Date:	**October 5, 2026**

I. BACKGROUND INFORMATION

A **random walk** is a mathematical process that describes a path consisting of a sequence of **random steps**. Random walks are used in many fields, such as physics, biology, economics, and computer science, to model systems that evolve randomly over time.

In this laboratory, we simulate multiple agents moving randomly on a grid using **AgentPy**, a Python library for **agent-based modeling**. Each agent represents an entity that can move one step at a time in a random direction. This simulation helps students understand:

- How **agents move and interact** in a grid-based environment.
- The concept of **bounded movement**, ensuring agents stay within the grid limits.
- What changing **parameters** like the number of agents, grid size, or number of steps affects the system.
- How **agent-based models** can be used to simulate real-world phenomena, such as **diffusion, crowd dynamics, or swarm behavior**.

II. TECHNICAL ASSESSMENT

RANDOM WALK SIMULATION USING AGENTPY

Objective

At the end of this laboratory activity, students are expected to:

1. To understand the concept of **random walks** in a grid environment.
2. To implement a **multi-agent simulation** using Python and AgentPy.
3. To visualize the **movement of agents** on a grid over time.

Materials Needed

- Computer with Python 3.x installed
- Libraries: agentpy, matplotlib
- Code editor (e.g., VSCode, PyCharm, or Jupyter Notebook)

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.

4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs in the Command Window.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA1_lastname.pdf**
   (Example: TA1_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Activity Instructions

Step 1: Run the Code

1. Open the provided Python code.

Random Walk Simulation (Console-Based)

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# -------- Get user input from console --------
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# -------- Agent Definition -----------
class RandomWalker(ap.Agent):
    def step(self):
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)

# -------- Model Definition -----------
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                              random.randint(0, self.p.grid_size[1]-1))

        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()

# -------- Parameters from user input -----------
```

```
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
}

# -------- Run Model -----------
model = RandomWalkModel(parameters)
model.setup()

# -------- Interactive Animation -----------
fig, ax = plt.subplots()
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0]+1))
ax.set_yticks(range(model.p.grid_size[1]+1))
ax.grid(True)

scat = ax.scatter([], [], s=200)  # s = size of agents

def update(frame):
    model.step()
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x, y)))
    return scat,

ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True, repeat=False)
plt.show()
```

2. Run the program in your terminal.
3. Enter the following values when prompted:
    - **Number of agents** (e.g., 5)
    - **Grid size** (e.g., 10 for a 10x10 grid)
    - **Number of steps** (e.g., 20)

These values control the number of agents, the size of the environment, and the simulation duration.

Step 2: Understand the Agent Behavior

1. Each agent is represented by a dot in the grid.
2. Agents move **randomly** in one of four directions: up, down, left, or right.
3. Agents **cannot leave the grid**; movement is restricted within the grid boundaries.

Step 3: Observe the Simulation

1. After running, a **grid with moving agents** is displayed.
2. Each time step updates the position of all agents.
3. Observe:
    - How agents spread across the grid over time
    - Whether agents cluster in certain areas or move freely

Step 4: Experiment with Parameters

- Repeat the simulation with **different values**:
  - **More agents** (e.g., 10 or 20)
  - **Smaller/larger grid sizes**
  - **Longer simulations** (e.g., 50 steps)
- Observe and record how the **agent movement patterns change**.

Laboratory Tasks

1. **Track agent paths**: Modify the code to **record and display the path of each agent** during the simulation.
2. **Change movement rules**: Make agents **avoid each other** or move in a **preferred direction** and observe the resulting patterns.
3. **Analyze final positions**: Record the **final positions** of all agents and analyze their **distribution**.
4. **Compare scenarios**: Run **two simulations with different parameters** and compare the movement patterns.

Modified (Updated Code - Laboratory Tasks)⇩

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation


# =============================================================
# GET USER INPUT
# =============================================================

num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

print("\nMovement Rules:")
print("1 - Random movement")
print("2 - Avoid other agents")
print("3 - Preferred direction (right)")

movement_choice = int(input("Choose movement rule: "))


# =============================================================
# AGENT DEFINITION
# =============================================================
```

```python
class RandomWalker(ap.Agent):

    def setup(self):
        # Random starting position
        self.position = (
            random.randint(0, self.model.p.grid_size[0] - 1),
            random.randint(0, self.model.p.grid_size[1] - 1)
        )

        # Store the agent's movement history
        self.path = [self.position]

    def step(self):

        # Possible movements
        directions = [
            (1, 0),    # Right
            (-1, 0),   # Left
            (0, 1),    # Up
            (0, -1)    # Down
        ]

        # ------------------------------------------------
        # RULE 1: RANDOM MOVEMENT
        # ------------------------------------------------

        if self.model.p.movement_rule == 1:

            direction = random.choice(directions)

            new_x = self.position[0] + direction[0]
            new_y = self.position[1] + direction[1]

            # Keep agent inside grid
            new_x = max(
                0,
                min(self.model.p.grid_size[0] - 1, new_x)
            )

            new_y = max(
                0,
                min(self.model.p.grid_size[1] - 1, new_y)
```

```
            )

            self.position = (new_x, new_y)

        # ----------------------------------------------------
        # RULE 2: AVOID OTHER AGENTS
        # ----------------------------------------------------

        elif self.model.p.movement_rule == 2:

            random.shuffle(directions)

            for direction in directions:

                new_x = self.position[0] + direction[0]
                new_y = self.position[1] + direction[1]

                # Check boundaries
                if not (
                    0 <= new_x < self.model.p.grid_size[0]
                    and
                    0 <= new_y < self.model.p.grid_size[1]
                ):
                    continue

                new_position = (new_x, new_y)

                # Check whether another agent occupies the cell
                occupied = False

                for other in self.model.agents:

                    if other is not self and other.position == new_position:
                        occupied = True
                        break

                # Move if the position is not occupied
                if not occupied:
                    self.position = new_position
                    break

        # ----------------------------------------------------
        # RULE 3: PREFERRED DIRECTION
```

```
        # ------------------------------------------------

        elif self.model.p.movement_rule == 3:

            # Agent prefers moving right
            preferred_direction = (1, 0)

            new_x = self.position[0] + preferred_direction[0]
            new_y = self.position[1] + preferred_direction[1]

            # If moving right would leave the grid,
            # choose a random valid direction instead.
            if new_x >= self.model.p.grid_size[0]:

                valid_directions = []

                for direction in directions:

                    test_x = self.position[0] + direction[0]
                    test_y = self.position[1] + direction[1]

                    if (
                        0 <= test_x < self.model.p.grid_size[0]
                        and
                        0 <= test_y < self.model.p.grid_size[1]
                    ):
                        valid_directions.append(direction)

                if valid_directions:
                    direction = random.choice(valid_directions)

                    new_x = self.position[0] + direction[0]
                    new_y = self.position[1] + direction[1]

            self.position = (new_x, new_y)

        # Record new position
        self.path.append(self.position)


# ============================================================
# MODEL DEFINITION
# ============================================================
```

```
class RandomWalkModel(ap.Model):

    def setup(self):

        # Create agents
        self.agents = ap.AgentList(
            self,
            self.p.agents,
            RandomWalker
        )

        # Create grid
        self.grid = ap.Grid(
            self,
            self.p.grid_size,
            torus=False
        )

        # Add agents to grid
        self.grid.add_agents(self.agents)

    def step(self):

        for agent in self.agents:
            agent.step()


# ============================================================
# PARAMETERS
# ============================================================

parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps,
    'movement_rule': movement_choice
}


# ============================================================
# CREATE MODEL
# ============================================================
```

```
model = RandomWalkModel(parameters)

model.setup()


# ============================================================
# MATPLOTLIB ANIMATION
# ============================================================

fig, ax = plt.subplots()

ax.set_xlim(-0.5, grid_size - 0.5)
ax.set_ylim(-0.5, grid_size - 0.5)

ax.set_xticks(range(grid_size))
ax.set_yticks(range(grid_size))

ax.grid(True)

ax.set_title("Random Walker Simulation")


# Agent dots
scat = ax.scatter([], [], s=200)


# Lines showing paths
path_lines = []

for agent in model.agents:

    line, = ax.plot([], [], linewidth=1)

    path_lines.append(line)


# ============================================================
# ANIMATION UPDATE
# ============================================================

def update(frame):
```

```
    # Move agents
    model.step()

    # Get current positions
    x = [
        agent.position[0]
        for agent in model.agents
    ]

    y = [
        agent.position[1]
        for agent in model.agents
    ]

    # Update agent dots
    scat.set_offsets(list(zip(x, y)))

    # Update paths
    for i, agent in enumerate(model.agents):

        path_x = [
            position[0]
            for position in agent.path
        ]

        path_y = [
            position[1]
            for position in agent.path
        ]

        path_lines[i].set_data(path_x, path_y)

    ax.set_title(
        f"Step {frame + 1} / {num_steps}"
    )

    return [scat] + path_lines


# ============================================================
# START ANIMATION
# ============================================================
```

```python
ani = animation.FuncAnimation(
    fig,
    update,
    frames=num_steps,
    blit=False,
    repeat=False
)

plt.show()


# ============================================================
# FINAL POSITION ANALYSIS
# ============================================================

print("\n====================================")
print("FINAL AGENT POSITIONS")
print("====================================")

for i, agent in enumerate(model.agents):

    print(
        f"Agent {i + 1}: {agent.position}"
    )


# ============================================================
# DISTRIBUTION ANALYSIS
# ============================================================

position_counts = {}

for agent in model.agents:

    position = agent.position

    if position not in position_counts:
        position_counts[position] = 0

    position_counts[position] += 1


print("\n====================================")
```

```
print("FINAL POSITION DISTRIBUTION")
print("====================================")

for position, count in position_counts.items():

    print(
        f"Position {position}: "
        f"{count} agent(s)"
    )


# ============================================================
# SUMMARY
# ============================================================

occupied_cells = len(position_counts)

print("\n====================================")
print("SUMMARY")
print("====================================")

print(f"Total agents: {num_agents}")
print(f"Grid size: {grid_size} x {grid_size}")
print(f"Simulation steps: {num_steps}")
print(f"Occupied cells: {occupied_cells}")

if occupied_cells < num_agents:
    print(
        "Some agents ended on the same grid cells."
    )
else:
    print(
        "Agents ended on separate grid cells."
    )
```

Step 5: Answer the Questions

1. What happens if the number of agents is **very high** compared to the grid size?
2. How does increasing the number of steps affect the **spread of agents**?
3. Do the agents ever move outside the grid? Why or why not?
4. How can this simulation be used to **model real-life phenomena** (e.g., diffusion, crowd movement)?

Answer the Questions ⇩

1. When number of agents is very high compared to the grid size, the cells are always occupied and the movements flash for a second before all agents occupy every cell.
2. It affects the overall spread of agents the more steps you input since it has more chances to end up in places further than its starting cell.
3. Agents do not move outside the grid because they are restricted to the boundaries of the grid. In the code, it checks each agent's new x and y coordinate and prevents both from becoming smaller than 0 or larger than the maximum grid coordinate.
4. These movements can represent situations where objects move randomly or according to simple movement rules in real life. These include crowd movement of people for example, or particles randomly moving and spread across everywhere.